Aula 04 – Modificando o Ambiente

Simulações Computacionais para o Ensino de Robótica Inteligente e Aprendizado por Reforço

Leonardo de Souza Gomes
Orientador: Prof. Dr. André Luiz C. Ottoni

Material adaptado do tutorial oficial Webots [1]
Edital PROGRAD nº 27, de 26 de maio de 2026
Universidade Federal de Ouro Preto

2026

Aula 04 – Modificando o Ambiente

Objetivo: aprender a modificar o ambiente de simulação no Webots, criando objetos simples que interagem fisicamente com o robô.

Ao final desta aula, você deverá entender:

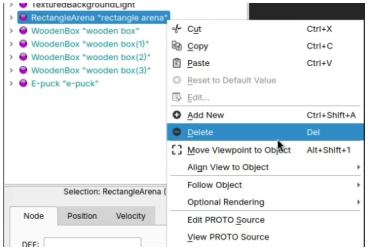
- o papel do nó `Solid` na representação de corpos rígidos;
- a diferença entre **representação gráfica** e **colisão física**;
- como adicionar a física de um objeto;
- como utilizar os mecanismos `DEF` e `USE`;

O tutorial em que essa aula se baseia pode ser encontrado em:
https://cyberbotics.com/doc/guide/tutorial-2-modification-of-the-environment?tab-language=c.

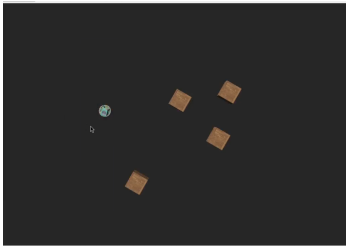
Substituindo o piso

O `RectangleArena` fornece um piso simples, fixado ao ambiente estático, e paredes ao redor. Nesta aula, vamos removê-lo e criar um piso manualmente.

1. Pause a simulação;
2. Reinicie a simulação;
3. Clique com o botão direito do mouse no node RectangleArena;
4. Clique em Delete, para remover o piso.

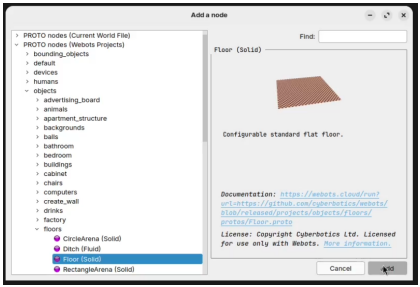


Resultado esperado:

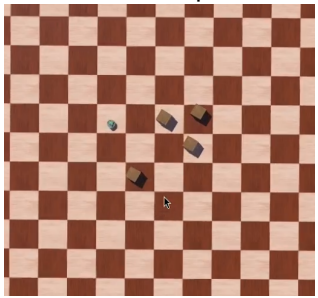


Substituindo o piso

1. Clique em Add new object;
2. Selecione o piso em: PROTO nodes > objects > floors > Floor (Solid);



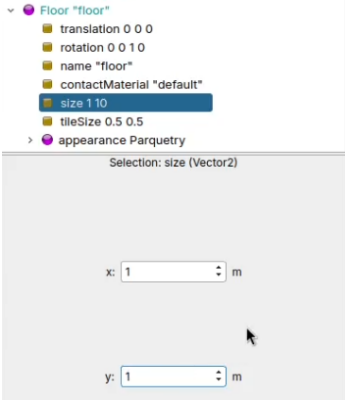
Resultado esperado:



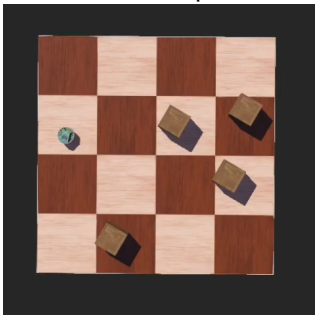
Modificando o tamanho do piso

Por padrão, este piso vem com a dimensão $10\,m \times 10\,m$, o que é muito grande para a escala do nosso projeto, por isso vamos diminuir o tamanho do piso para $1\,m \times 1\,m$.

1. Clique em Floor;
2. Modifique o field size para x = 1m e y = 1m;



Resultado esperado:



O `Solid` Node

`Solid` é um dos principais nós-base do Webots, é a partir dele que podemos fazer estruturas mais complexas combinando-os.

Corpo rígido

Um corpo rígido é um corpo cuja deformação pode ser desprezada. A distância entre dois pontos do corpo permanece constante ao longo do tempo. Ou seja, não possui partes móveis.

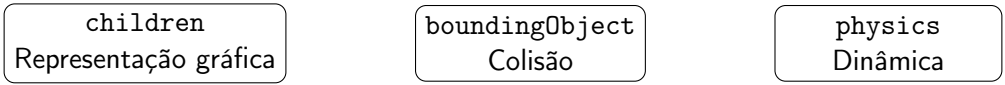
Não são corpos rígidos: cordas, braços robóticos e estruturas articuladas consideradas como um único corpo.

Estrutura de um corpo rígido

O motor de física do Webots é voltado para a simulação de corpos rígidos.

Um `Solid` pode ser entendido a partir de três componentes principais:

1. Children: Representação gráfica, define como o objeto aparece visualmente;
2. BoundingObject: Colisão, define seus limites para colisão;
3. Physics: Dinâmica, determina se o objeto participa da dinâmica física.



Shape e Geometry

Um objeto pode possuir uma representação gráfica e uma forma física de colisão.

Shape

Organiza a aparência de uma geometria, podendo conter campos como `appearance` e `geometry`.

Geometry

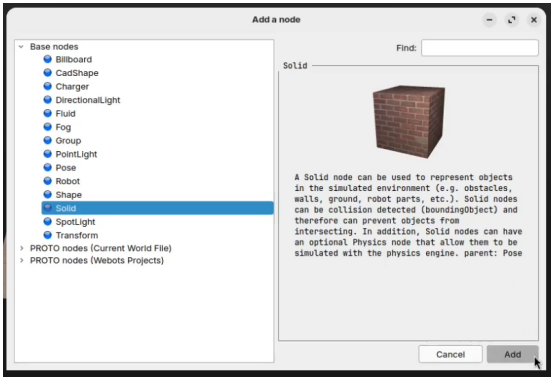
Representa uma primitiva geométrica, como: Sphere, Box ou Cylinder.

A mesma geometria pode aparecer tanto no contexto gráfico quanto no contexto físico, desde que seja suportada pelo campo correspondente.

Criando uma esfera

Agora vamos criar um corpo rígido do tipo esfera.

1. Adicione um corpo rígido: Add a new object > Base nodes > Solid;

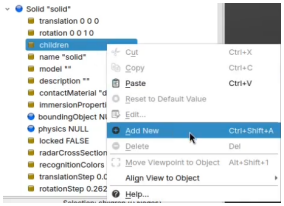


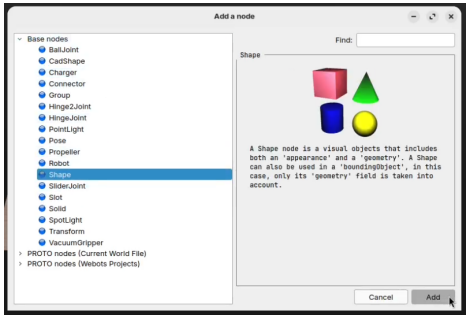
Importante

Não aparecerá nada ainda da renderização 3D, pois não temos um Shape.

Configurando a esfera

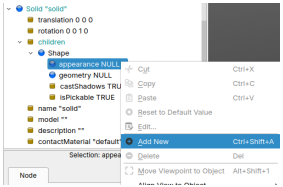
1. Adicione um Shape: Solid > children > Add New > Shape.

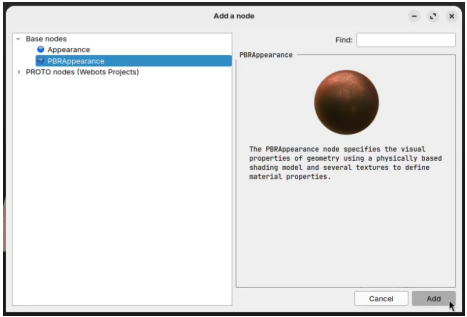




Configurando a esfera

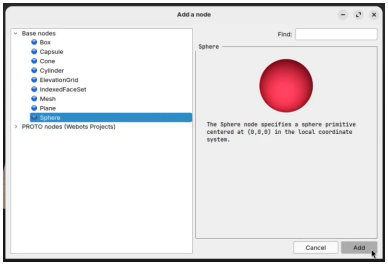
1. Abra, Solid > children > Shape > appearance.
2. Adicione uma aparência: Base nodes > PBRAppearance.



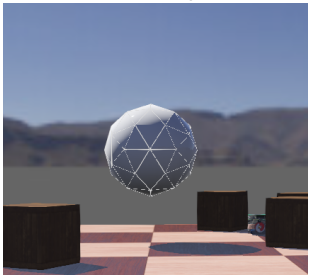


Configurando a esfera

1. Abra, Solid > children > Shape > geometry;
2. Adicione uma geometria: Base nodes > Sphere;
3. Em `PBRAppearance`, defina: metalness = 0;
4. Altere a posição do `Solid` para: translation = {0.2, 0.2}.

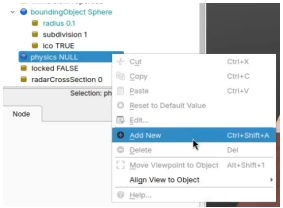


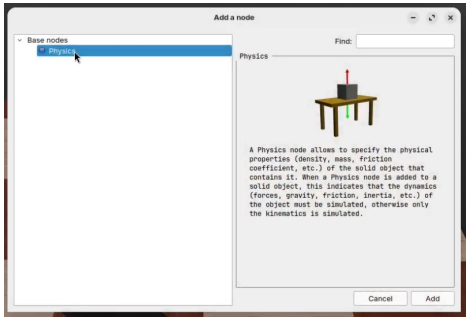
Resultado esperado:



Configurando a esfera

1. Abra, physics > Add New.
2. Adicione a física: Base nodes > physics.
3. salve o mundo.

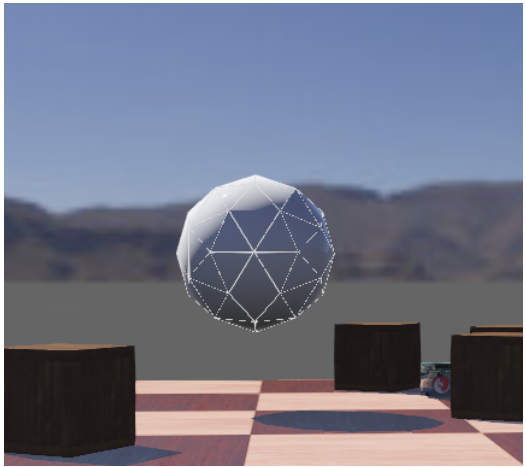


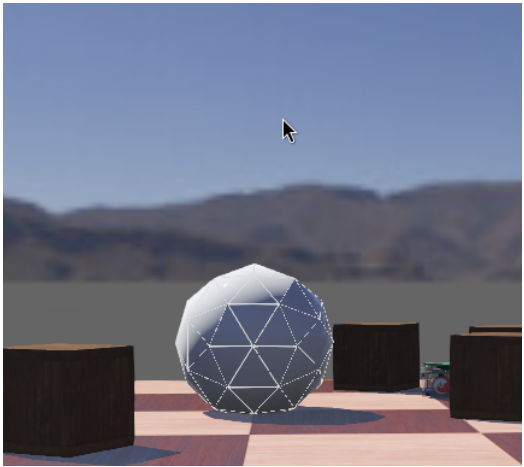


O que acontece ao executar?

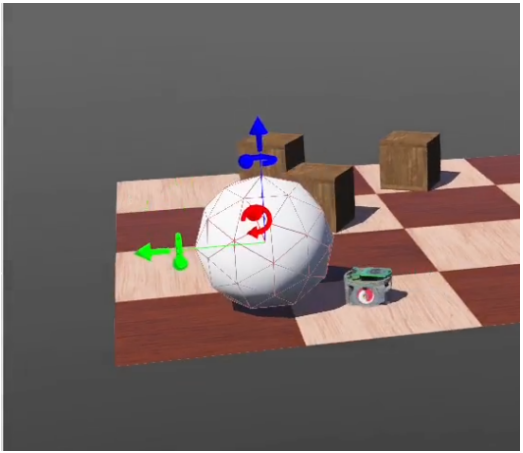
Como o `Solid` possui um nó `Physics`, a esfera participa da simulação dinâmica e sofre a ação da gravidade.

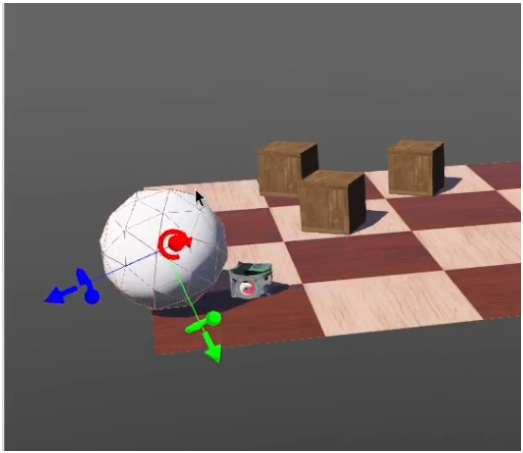
Simulação da Física





Simulação da Física

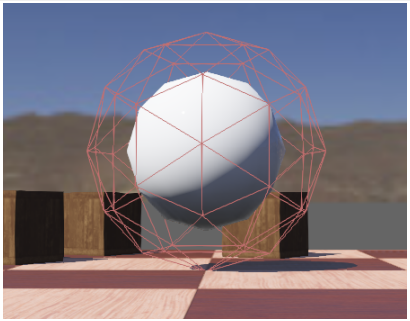




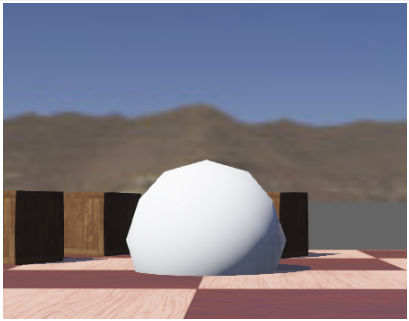
Para que o robô colida com a bola, deve-se usar o controller epuck_go_forward.

Atenção

Para o nosso exemplo, o `boundingObject` e o `Shape > geometry` devem possuir o mesmo `radius`. Se o `boundingObject` for maior, a esfera parecerá flutuar.
Se o `Shape > geometry` for maior, a esfera poderá visualmente atravessar outros objetos antes da colisão.



`boundingObject: radius = 0.15`
`Shape > geometry: radius = 0.1`



`boundingObject: radius = 0.05`
`Shape > geometry: radius = 0.1`

O Mecanismo DEF-USE

O mecanismo DEF-USE permite definir um nó uma única vez e reutilizar sua definição em outros pontos da árvore da cena.

DEF

Dá um nome a uma definição:

```
DEF BALL_GEOMETRY Sphere{...}
```

USE

Reutiliza uma definição existente:

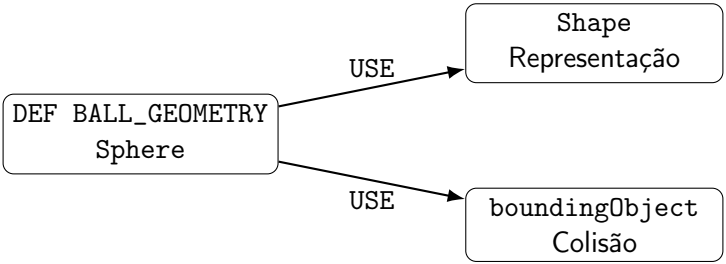
```
USE BALL_GEOMETRY
```

Vantagens:

- Reduz a duplicação no arquivo do mundo;
- Facilita alterações em objetos repetidos;
- Permite reutilizar a mesma definição em diferentes locais.

Benefício do DEF-USE

Depois de utilizar DEF-USE, alterar a definição original também altera os locais que a reutilizam.



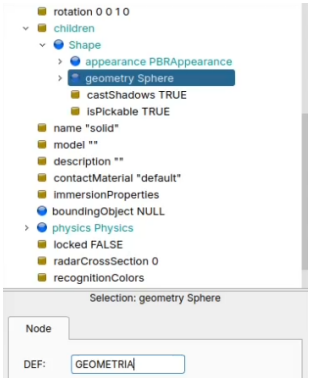
Por exemplo, modificar o radius da esfera altera sua definição reutilizada nos dois contextos.

DEF-USE: exemplo com a esfera

Inicialmente, temos duas `Esferas` iguais: uma para a aparência e outra para a colisão.

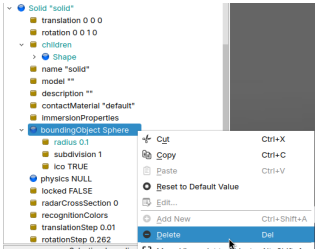
Podemos definir a primeira como:

```
DEF GEOMETRIA Sphere{...}
```

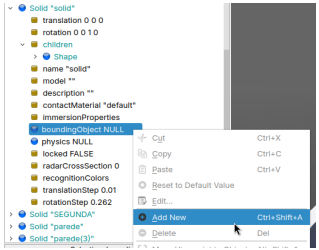


DEF-USE: exemplo com a esfera

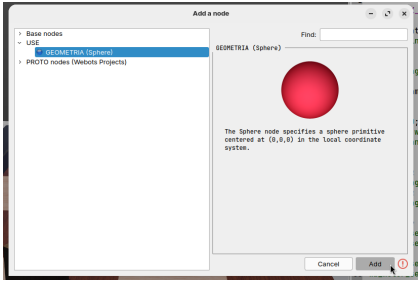
1. Apague o `boundingObject`;



2. Adicione um novo `boundingObject`;

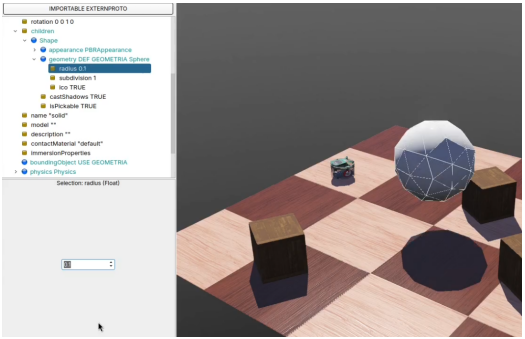


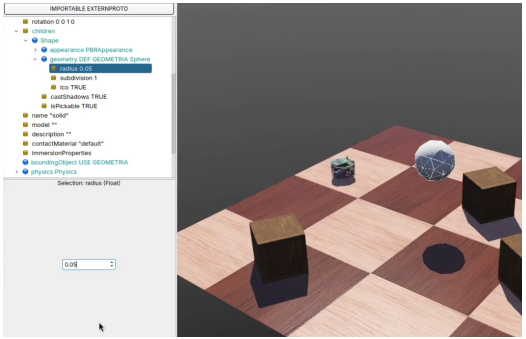
3. Utilize a definição no `boundingObject`: `USE GEOMETRIA`;



DEF-USE: exemplo com a esfera

Com isso, ao modificarmos o campo: children > Shape > geometry > radius, o `boundingObject` muda automaticamente;





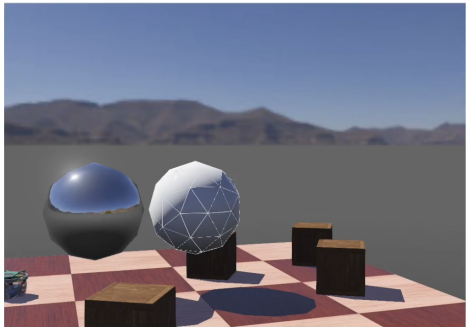
Atividade prática

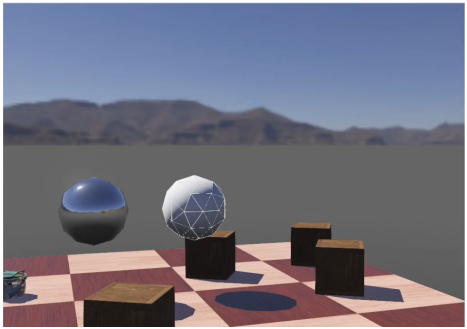
O mecanismo também pode ser aplicado ao nó `Shape`. Crie uma segunda esfera com os mesmos parâmetros, utilizando `DEF-USE` em `Shape` e `boundingObject`.

DEF-USE no nível de Shape

Por que utilizar Shape?

O Shape pode ser reutilizado diretamente para fins gráficos e também pode ser colocado no campo `boundingObject`.





Essa estratégia será especialmente útil quando vários objetos precisarem compartilhar a mesma geometria e aparência. Para fins didáticos, não aplicamos a definição ao Shape da segunda esfera.

Adicionando paredes

Agora vamos construir quatro paredes para cercar o ambiente.

Atividade prática

Crie quatro paredes:

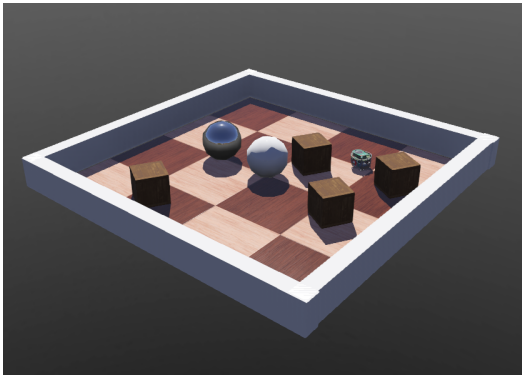
- utilizando o nó `Box` como geometria;
- sem adicionar `Physics`;



Adicionando paredes

Resultado esperado:





Objetos estáticos e dinâmicos

A presença ou ausência do nó `Physics` determina o comportamento do objeto na simulação.

Estático

`physics = NULL`

O objeto permanece fixo no ambiente, mesmo que esteja no ar.

Dinâmico

`physics = Physics{}`

O objeto participa da simulação física e pode sofrer a ação da gravidade.

No exemplo

A esfera é dinâmica; as paredes devem ser estáticas.

Agradecimentos

Este trabalho foi realizado com o apoio da Pró-Reitoria de Graduação (PROGRAD), do Laboratório Machine Learning and Robotics (MLBots) e do Departamento de Computação (DECOM) da Universidade Federal de Ouro Preto (UFOP).

Leonardo de Souza Gomes, leonardo.sg@aluno.ufop.br

Prof. Dr. André Luiz C. Ottoni, andre.ottoni@ufop.edu.br

Edital PROGRAD nº 27, de 26 de maio de 2026







Referências

[1] Cyberbotics, *Tutorial 2: Modification of the Environment*, https://cyberbotics.com/doc/guide/tutorial-2-modification-of-the-environment?tab-language=c, Acesso em: 20 ago. 2026, 2026. (acesso em 20/08/2026).